

DevOps avancé
Année universitaire 2026–2027

Travaux pratiques, TP1

Continuous Deployment avec GitOps et ArgoCD

Un dépôt Git décrit l'état désiré du cluster. ArgoCD le surveille, l'applique, corrige toute dérive et permet de revenir à une version stable.

Réalisé par

Kacem Mathlouthi
Oussema Kraiem
Med Amin Haouas

Dépôt

`github.com/KacemMathlouthi/tp-devops`

Image

`ghcr.io/kacemmathlouthi/tp-devops`

Table des matières

Présentation générale 3
Objectif 3
Application support 3
Construction de l'image 3
Stack technique 4
Architecture 4
Étape 1 : structure du dépôt Git 5
Organisation 5
Base Kustomize 5
Overlay `dev` 6
Étape 2 : installation et configuration d'ArgoCD 7
Cluster Kubernetes 7
Installation d'ArgoCD 7
Accès à l'interface 8
Création de l'Application 9
Démonstration 1 : déploiement continu automatisé 12
Préparation : construction de la version v2 12
Modification de l'état désiré dans Git 12
Synchronisation automatique par ArgoCD 12
Démonstration 2 : détection du drift et self-healing 14
Démonstration 3 : Sync Status et Health Status 15
Démonstration 4 : historique et rollback 16
Conclusion 17

Présentation générale

Objectif

Ce TP met en place un pipeline GitOps de bout en bout : l'état désiré de l'application et de ses ressources Kubernetes est décrit de façon déclarative dans un dépôt Git, et ArgoCD, installé dans le cluster, se charge de le faire converger vers cet état. Aucun `kubectl apply` n'est lancé depuis l'extérieur pour déployer : le cluster tire (pull) sa configuration depuis Git.

Le TP doit démontrer cinq concepts :

Single Source of Truth	Tout l'état désiré vit dans Git : manifests Kustomize et définition de l'Application ArgoCD.
Pull-based CD	ArgoCD surveille le dépôt et applique automatiquement chaque commit.
Drift et self-healing	Toute modification manuelle du cluster est détectée puis annulée.
Sync et Health	Un manifeste conforme à Git (Synced) peut produire des pods en échec (Degraded).
Rollback et historique	Retour instantané à une révision stable depuis l'onglet History and Rollback.

Application support

Le sujet laissant le choix de l'application, nous avons retenu une petite API `FastAPI` (Python 3.12, dépendances gérées par `uv`) qui expose trois routes :

- `GET /` : page HTML affichant la version déployée et le nom du pod qui a servi la requête ;
- `GET /version` : la même information au format JSON ;
- `GET /healthz` : sonde utilisée par Kubernetes (readiness et liveness).

La version affichée est injectée au build de l'image (`ARG APP_VERSION`) : chaque démonstration est ainsi visible directement dans le navigateur, et le nom du pod permet de constater la répartition de charge entre réplicas.

Construction de l'image

Le dépôt contient le code applicatif (`app/`) et une GitHub Action (`.github/workflows/build.yml`) qui, à chaque tag Git `v*`, construit l'image et la publie sur GitHub Container Registry :

```
git tag v2 && git push origin v2
# → ghcr.io/kacemmathlouthi/tp-devops:v2
```

Séparation CI / CD

La CI (GitHub Actions) se contente de produire une image versionnée ; elle n'a aucun accès au cluster. Le déploiement (CD) est entièrement piloté par ArgoCD, qui tire l'état désiré depuis Git : changer la version déployée revient à modifier le tag dans l'overlay Kustomize et à pousser le commit.

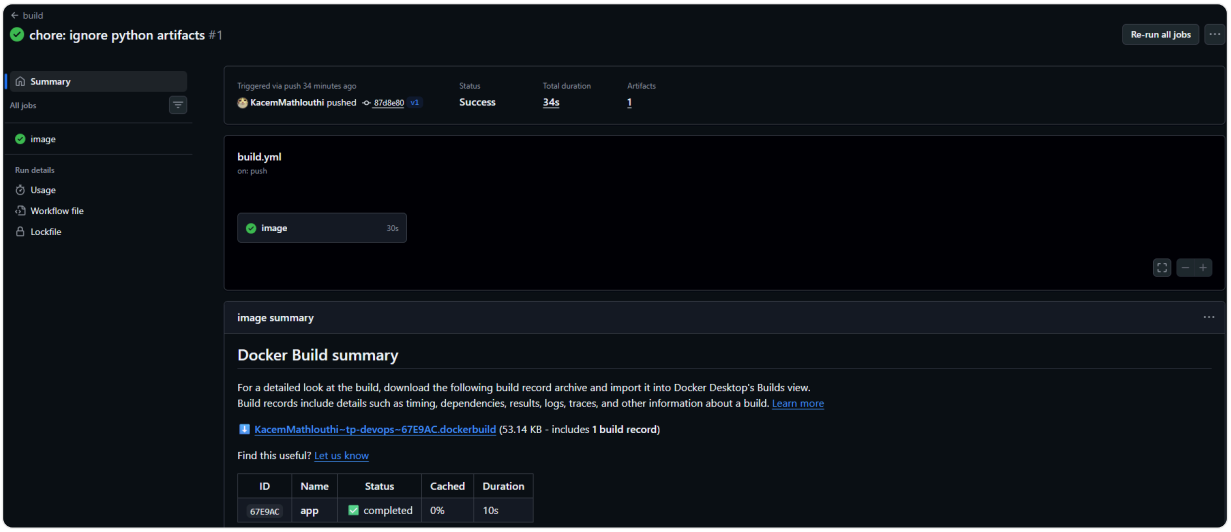
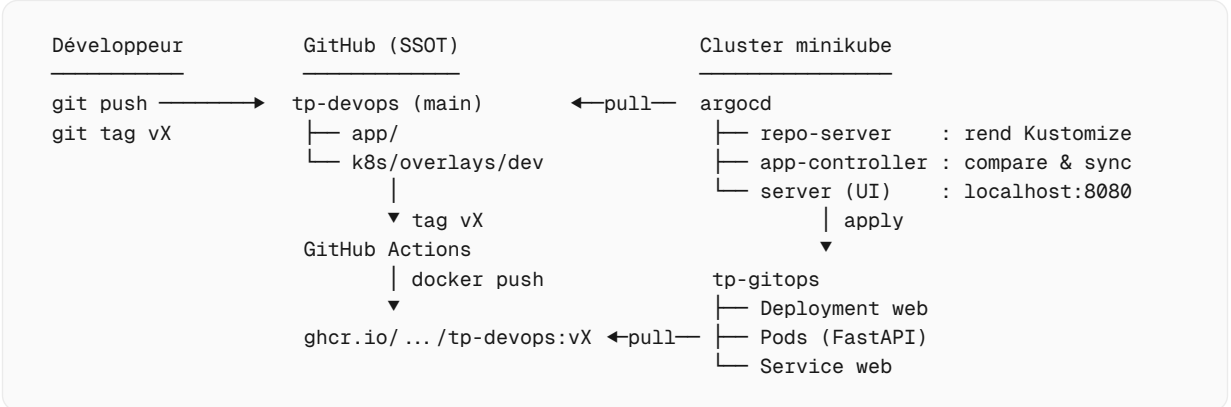


Fig. 1. GitHub Actions : le push du tag `v1` déclenche le workflow `build` (34 s) qui construit et publie l'image

Stack technique

Cluster	minikube (driver Docker) sous WSL 2
GitOps	ArgoCD (manifests officiels, namespace `argocd`)
Manifests	Kustomize : une `base` et un overlay `dev`
Dépôt	GitHub public `KacemMathlouthi/tp-devops`, branche `main`
Application	FastAPI (Python 3.12, uv), image Docker non-root
CI	GitHub Actions vers `ghcr.io/kacemmathlouthi/tp-devops:`

Architecture



Étape 1 : structure du dépôt Git

Organisation

Un seul dépôt contient l'ensemble du TP : code applicatif, pipeline de build et manifests. ArgoCD ne lit que le chemin `k8s/overlays/dev` ; le reste du dépôt (code, rapport, définition de l'Application) ne perturbe donc pas la synchronisation.

```
tp-devops/
├── app/                          # code applicatif
│   ├── main.py                   # API FastAPI
│   ├── pyproject.toml, uv.lock   # dépendances (uv)
│   └── Dockerfile
├── .github/workflows/build.yml   # build + push de l'image sur tag vX
├── k8s/
│   ├── base/                     # ressources communes
│   │   ├── kustomization.yaml
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   └── overlays/
│       └── dev/
│           └── kustomization.yaml  # namespace, replicas, tag d'image
├── argocd/
│   └── application.yaml          # l'Application ArgoCD (déclarative elle aussi)
└── compte-rendu/                 # ce rapport
```

Base Kustomize

La base décrit l'application indépendamment de tout environnement : un `Deployment` et un `Service`, auxquels Kustomize ajoute des labels communs.

```
# k8s/base/kustomization.yaml
labels:
  - pairs:
      app.kubernetes.io/name: web
      app.kubernetes.io/part-of: tp-gitops
resources:
  - deployment.yaml
  - service.yaml
```

Points notables du `Deployment` :

- sondes `readinessProbe` et `livenessProbe` sur `/healthz` : c'est sur elles que repose le Health Status calculé par ArgoCD ;
- variable `POD_NAME` injectée via l'API downward, affichée par l'application ;
- requests/limits CPU et mémoire ;
- `progressDeadlineSeconds: 60` : en cas d'image introuvable, Kubernetes marque le rollout en échec au bout d'une minute (au lieu de 10 par défaut) et ArgoCD affiche alors Degraded.

Le `Service` (`ClusterIP`, port 80 → 8000) répartit le trafic entre les pods portant le label `app: web`.

Overlay `dev`

L'overlay est le seul fichier modifié pendant les démonstrations : il fixe le namespace, le nombre de réplicas et le tag de l'image.

```
# k8s/overlays/dev/kustomization.yaml
namespace: tp-gitops
resources:
  - ../../base
replicas:
  - name: web
    count: 2
images:
  - name: ghcr.io/kacemmathlouthi/tp-devops
    newTag: v1
```

Le rendu final peut être vérifié localement sans cluster :

```
kubectl kustomize k8s/overlays/dev
```

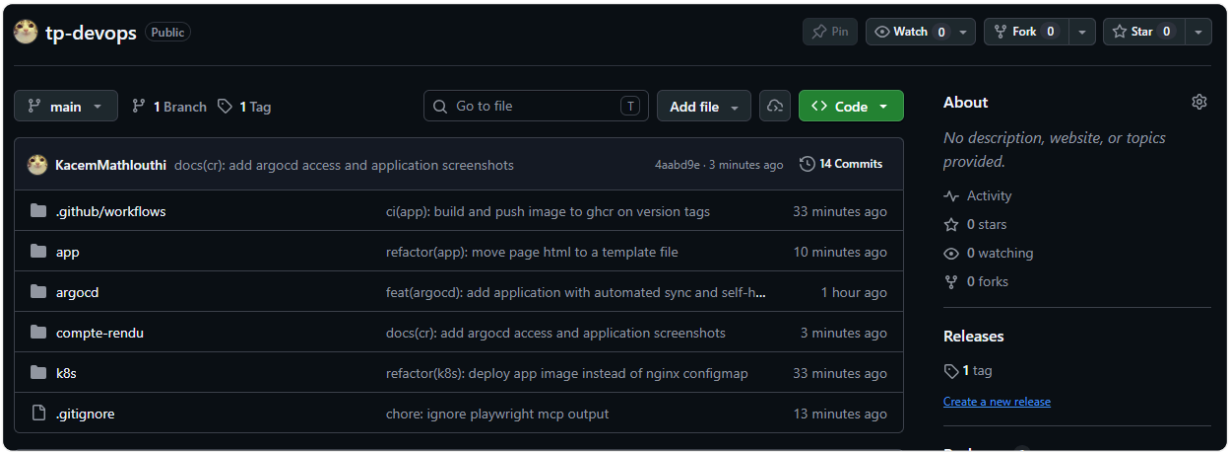


Fig. 2. Dépôt public `KacemMathlouthi/tp-devops` : code applicatif, workflow CI, manifests Kustomize, Application ArgoCD et rapport ; le package GHCR `tp-devops` est lié au dépôt

Étape 2 : installation et configuration d'ArgoCD

Cluster Kubernetes

Le cluster est un minikube mono-nœud utilisant le driver Docker, sous WSL 2. Deux vCPU et environ 4 Go de RAM suffisent pour Kubernetes, ArgoCD (7 pods) et l'application.

```
minikube start --driver=docker --cpus=2 --memory=4g
kubectl config use-context minikube
kubectl get nodes
```

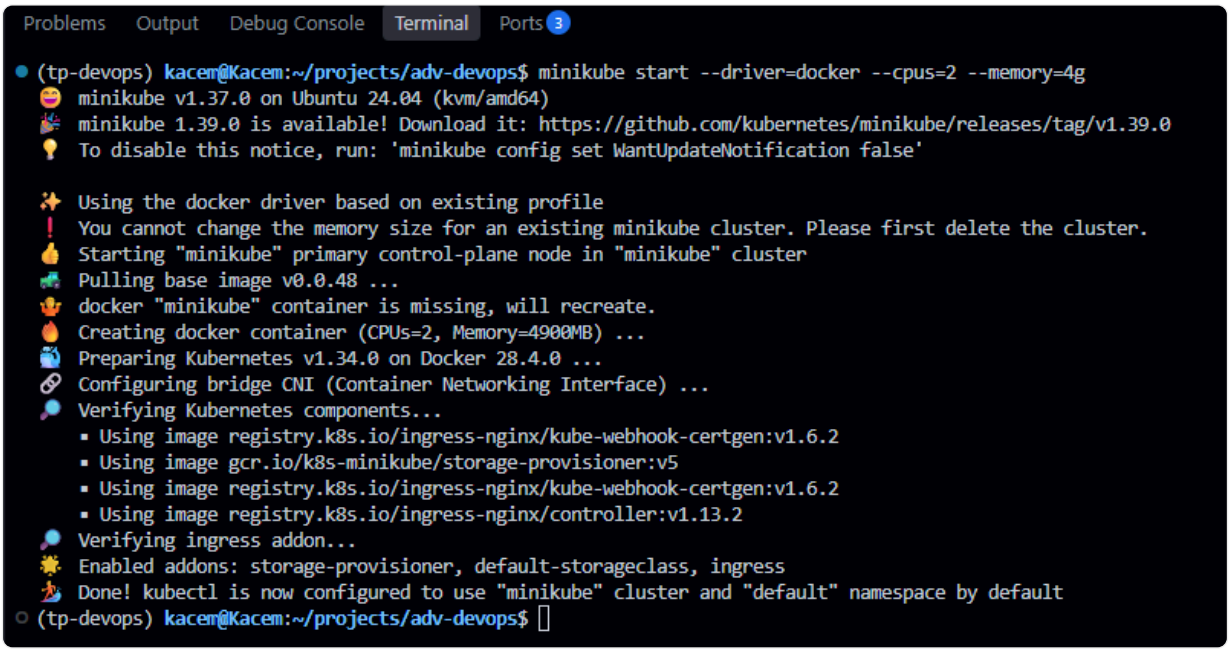


Fig. 3. Démarrage du cluster minikube (Kubernetes v1.34.0, driver Docker) ; `kubectl` est configuré sur le contexte `minikube`

Installation d'ArgoCD

ArgoCD est installé à partir des manifests officiels dans son propre namespace. L'option `--server-side` est nécessaire car certaines CRD dépassent la taille maximale de l'annotation `last-applied-configuration`.

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
```

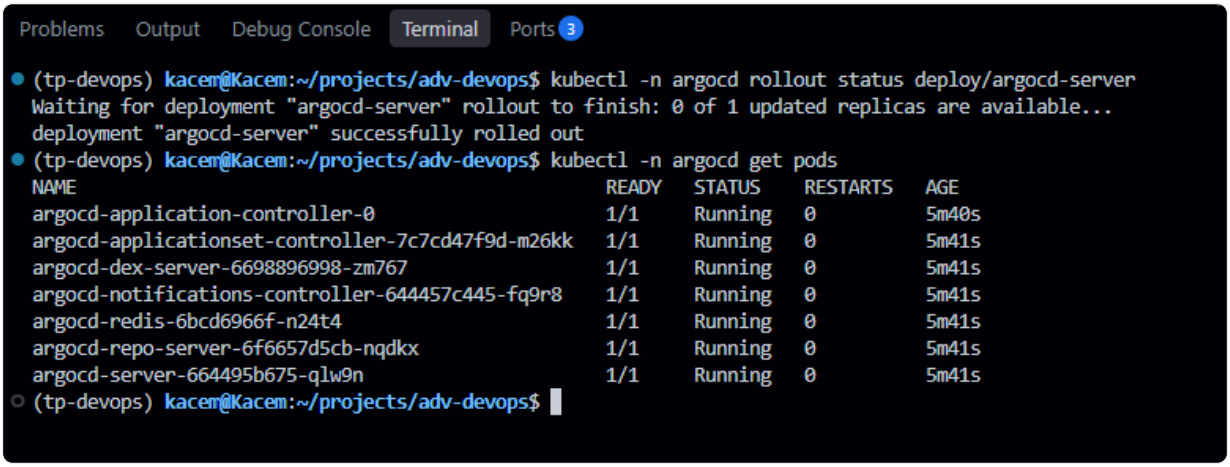


Fig. 4. Fin du rollout de `argocd-server` puis les 7 pods ArgoCD en état `Running` : application-controller, applicationset-controller, dex, notifications, redis, repo-server et server

Accès à l'interface

```
# mot de passe initial de l'utilisateur admin
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d; echo
# exposition locale de l'UI
kubectl -n argocd port-forward svc/argocd-server 8080:443
```

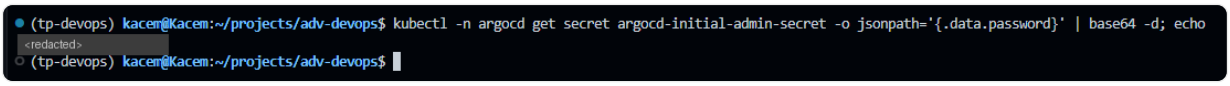


Fig. 5. Récupération du mot de passe initial de l'utilisateur `admin` (valeur masquée)

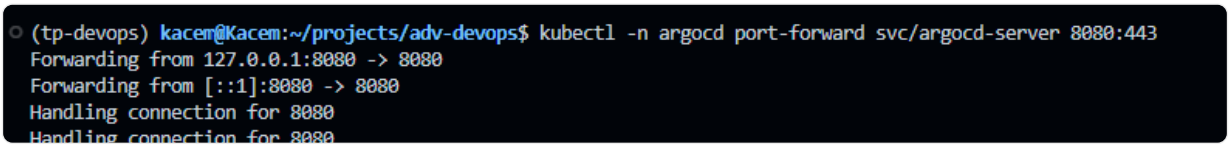


Fig. 6. Port-forward du service `argocd-server` vers `localhost:8080`

L'interface est alors accessible sur `https://localhost:8080` (certificat auto-signé). Après connexion, ArgoCD (v3.5.5) ne gère encore aucune application.

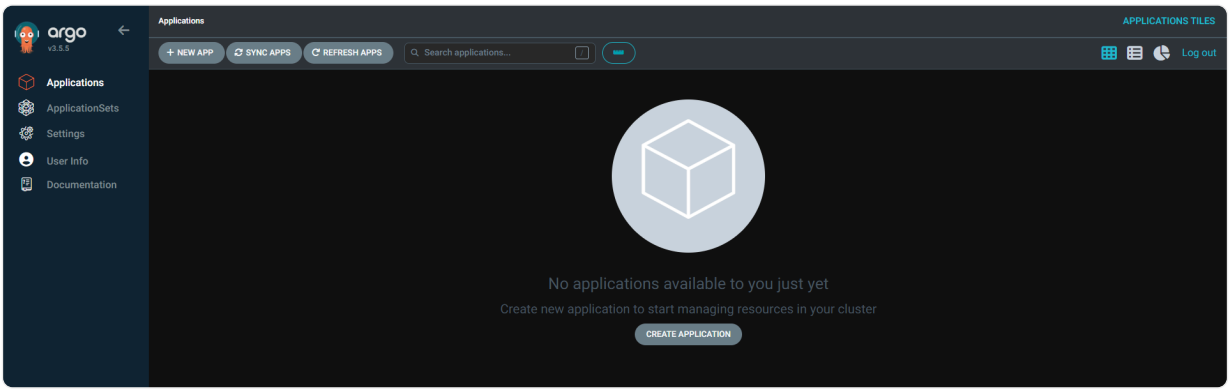


Fig. 7. Tableau de bord ArgoCD après la première connexion : aucune application

Création de l'Application

L'Application est elle-même décrite dans Git (`argocd/application.yaml`) : la configuration de l'outil GitOps suit le même principe déclaratif que l'application.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: tp-gitops
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/KacemMathlouthi/tp-devops.git
    targetRevision: main
    path: k8s/overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: tp-gitops
  syncPolicy:
    automated:
      prune: true      # supprime ce qui a disparu de Git
      selfHeal: true   # annule les modifications manuelles
    syncOptions:
      - CreateNamespace=true
```

```
kubectl apply -f argocd/application.yaml
kubectl -n tp-gitops get pods -w
```

C'est la seule commande `kubectl apply` du TP : à partir de là, ArgoCD crée lui-même le namespace `tp-gitops` et les ressources décrites dans Git. Les deux pods passent de `ContainerCreating` à `Running` puis `1/1` (readiness probe validée) en moins de 30 secondes.

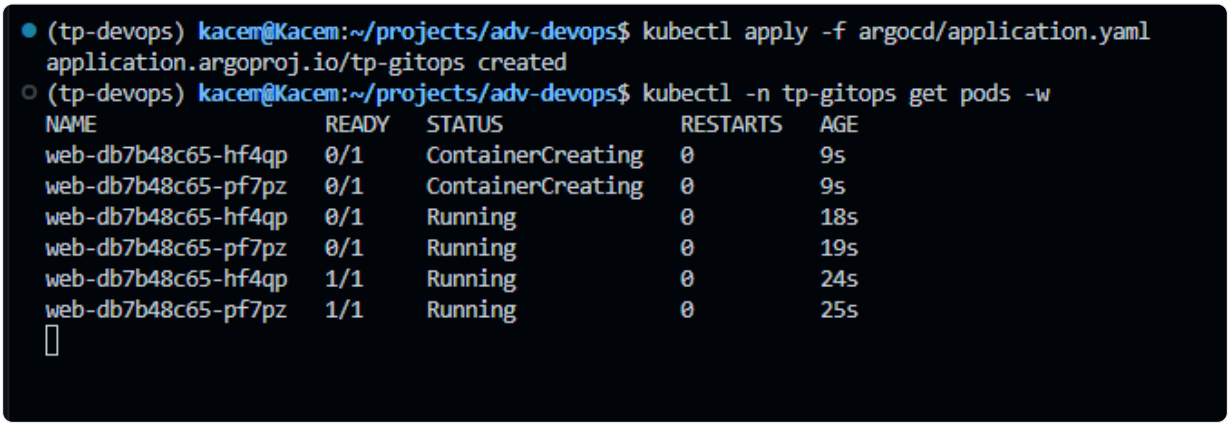


Fig. 8. Création de l'Application puis démarrage des deux pods `web` déployés par ArgoCD

Options de synchronisation

- **automated** : ArgoCD synchronise dès qu'un nouveau commit est détecté, sans clic sur Sync.
- **selfHeal** : si l'état du cluster s'écarte de Git (drift), ArgoCD réapplique les manifests.
- **prune** : une ressource retirée de Git est supprimée du cluster.

– ArgoCD interroge Git toutes les 3 minutes par défaut ; le bouton Refresh force une vérification immédiate.

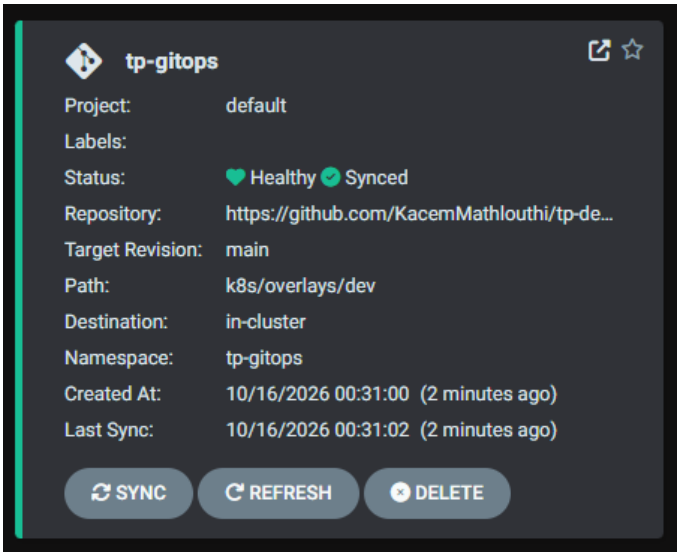


Fig. 9. Tuile de l'application `tp-gitops` : dépôt, révision `main`, chemin `k8s/overlays/dev`, statut Healthy et Synced

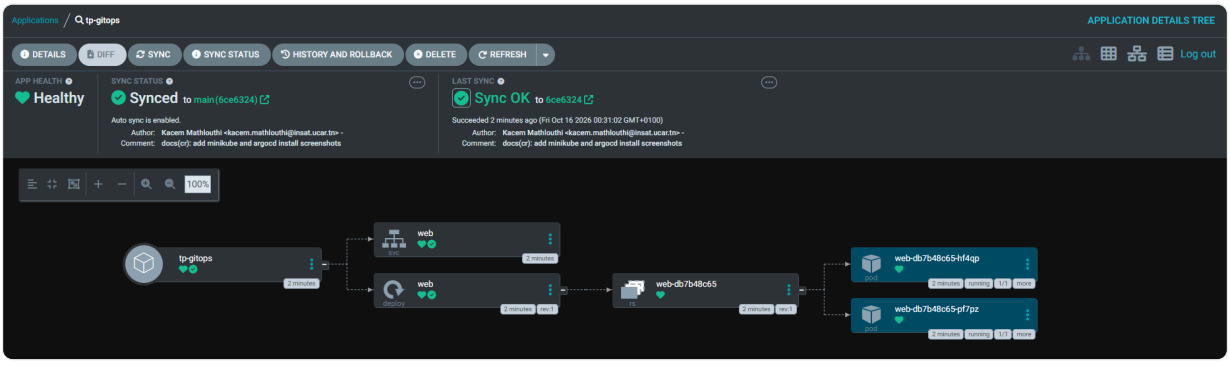


Fig. 10. Arbre des ressources : Service et Deployment `web`, ReplicaSet et deux pods. Synchronisé automatiquement sur le commit `6ce6324` (Auto sync is enabled)

L'en-tête de la vue détaillée illustre déjà la distinction au cœur du TP : App Health (état réel des ressources, ici Healthy) et Sync Status (conformité avec Git, ici Synced sur `main`) sont deux indicateurs indépendants.

```
kubectl -n tp-gitops port-forward svc/web 8081:80   # http://localhost:8081
```



Fig. 11. Application servie par le cluster : version `v1`, réponse du pod `web-db7b48c65-hf4qp`

Démonstration 1 : déploiement continu automatisé

Objectif : montrer qu'un simple commit suffit à modifier le cluster, sans aucune commande `kubectl`.

Préparation : construction de la version v2

La nouvelle version de l'application (refonte de la page) est publiée en poussant un tag Git. La CI produit l'image `ghcr.io/kacemmathlouthi/tp-devops:v2` mais ne touche pas au cluster : tant que Git n'y fait pas référence, rien n'est déployé.

```
git tag v2 && git push origin v2
```

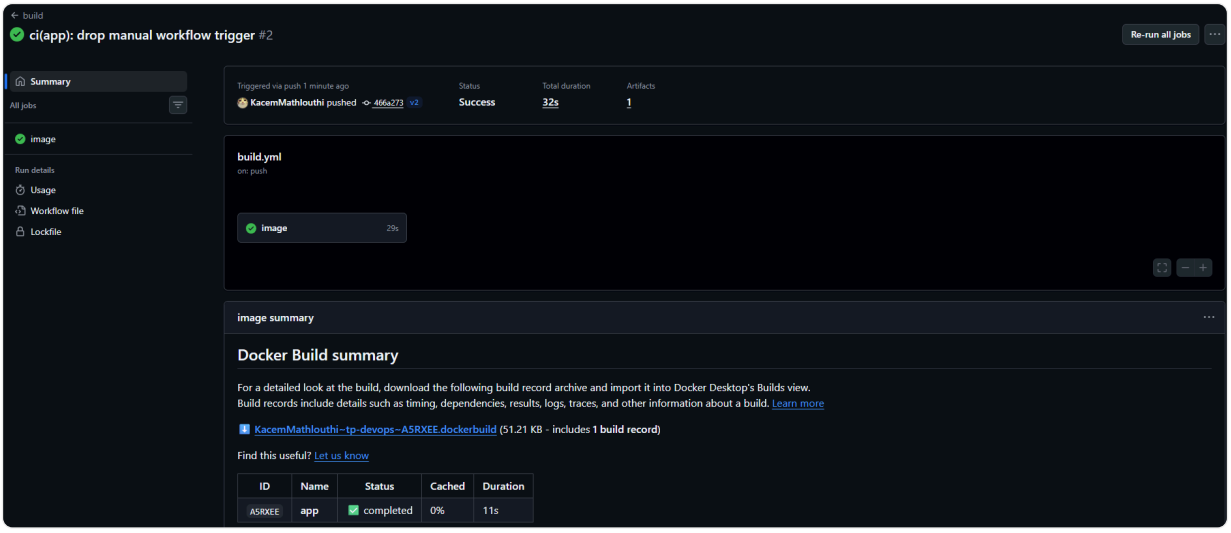


Fig. 12. GitHub Actions : le tag `v2` déclenche le build et la publication de l'image `tp-devops:v2` (32 s)

Modification de l'état désiré dans Git

Le déploiement de la nouvelle version se résume à changer une ligne dans l'overlay, puis à pousser le commit. Aucune commande n'est envoyée au cluster.

```
# k8s/overlays/dev/kustomization.yaml
 images:
   - name: ghcr.io/kacemmathlouthi/tp-devops
-    newTag: v1
+    newTag: v2
```

```
git commit -am "feat(k8s): deploy v2" && git push
```

Synchronisation automatique par ArgoCD

ArgoCD interroge le dépôt périodiquement (toutes les 3 minutes par défaut) ; le bouton Refresh force une vérification immédiate sans déclencher de synchronisation manuelle. Dès que le nouveau commit

est détecté, la politique `automated` applique le Deployment modifié et Kubernetes effectue un rolling update.

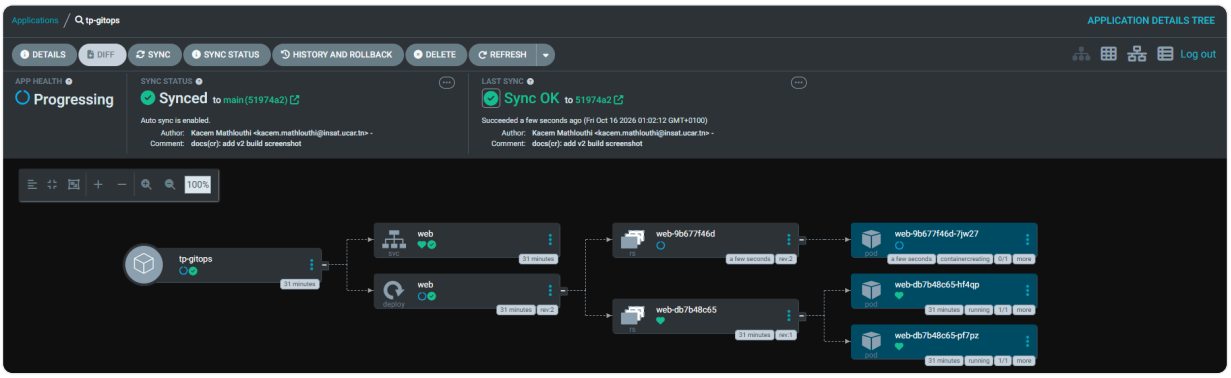


Fig. 13. Pendant la synchronisation : l'application est Synced sur le nouveau commit mais Progressing. Un nouveau ReplicaSet `web-9b677f46d` (`rev:2`, image v2) démarre son premier pod (`ContainerCreating`) tandis que les deux pods v1 continuent de servir le trafic

Le rolling update respecte les valeurs par défaut du Deployment (`maxSurge: 25%`, `maxUnavailable: 25%`) : un pod v2 est créé, et ce n'est qu'une fois sa readiness probe validée qu'un pod v1 est supprimé. La capacité ne descend jamais sous deux pods prêts.

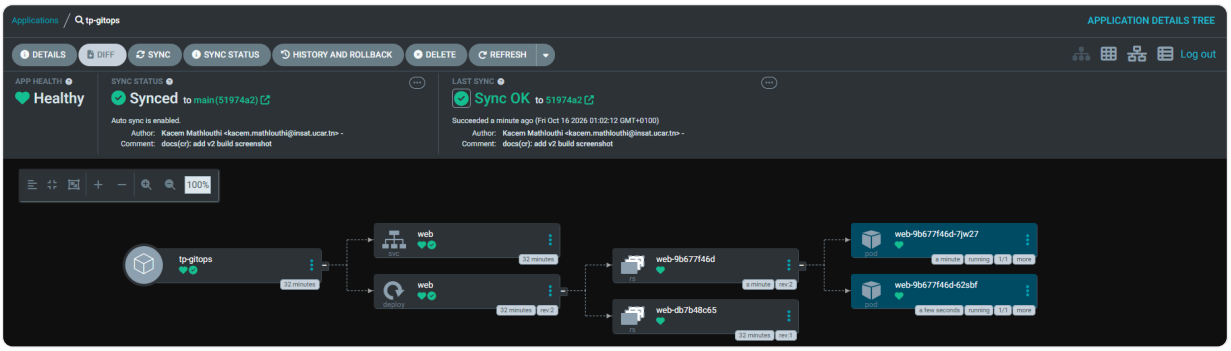


Fig. 14. Fin du rollout : Healthy et Synced. Les deux pods appartiennent au ReplicaSet v2 ; l'ancien ReplicaSet `web-db7b48c65` (`rev:1`) est conservé à 0 réplica comme historique Kubernetes (`revisionHistoryLimit`)

Capture à ajouter

Fig. 15. Page de l'application en version v2

- **Résultat**

Un simple commit dans Git a suffi à déployer la version v2 : ArgoCD a détecté l'écart entre l'état désiré et l'état réel, puis a appliqué le changement sans aucune intervention sur le cluster. Le statut de santé est passé de Progressing à Healthy à la fin du rolling update, sans interruption de service.

Démonstration 2 : détection du drift et self-healing

Objectif : altérer manuellement le cluster et montrer qu'ArgoCD rétablit l'état décrit dans Git.

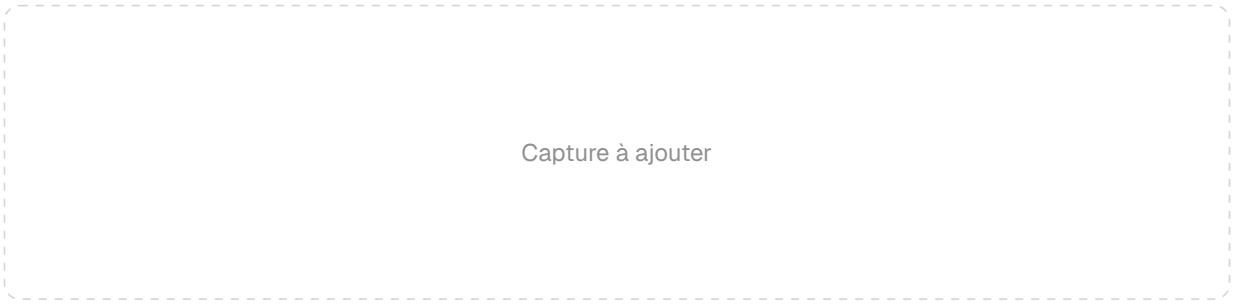


Fig. 16. Suppression manuelle du Deployment avec `kubectl delete`

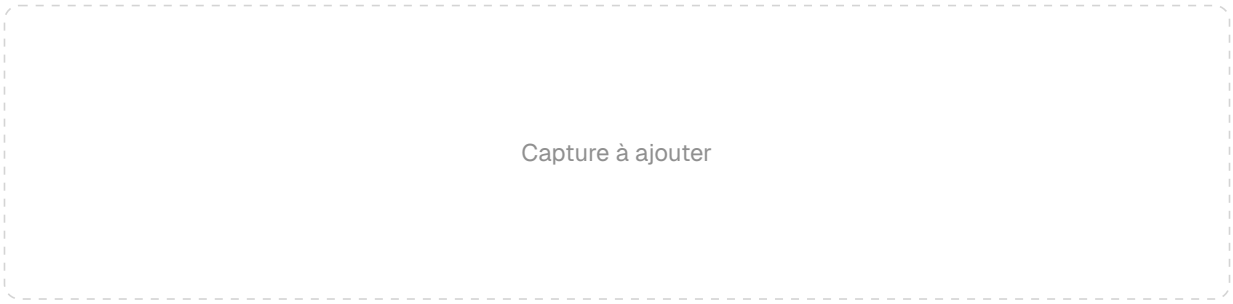


Fig. 17. ArgoCD : drift détecté (OutOfSync) puis réconciliation

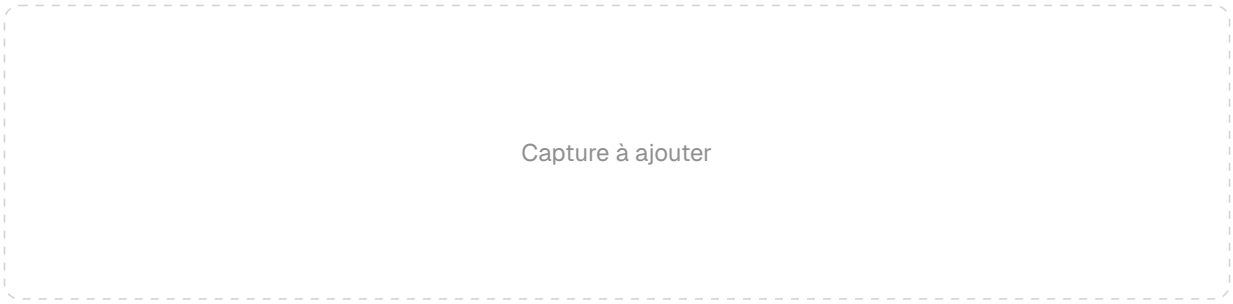


Fig. 18. Le Deployment est recréé automatiquement

Résultat

À compléter après la démonstration.

Démonstration 3 : Sync Status et Health Status

Objectif : montrer qu'une application peut être parfaitement synchronisée avec Git tout en étant défaillante.

Capture à ajouter

Fig. 19. Commit introduisant un tag d'image inexistant

Capture à ajouter

Fig. 20. ArgoCD : application Synced mais Degraded

Capture à ajouter

Fig. 21. Pod en `ErrImagePull` / `ImagePullBackOff`

Résultat

À compléter après la démonstration.

Démonstration 4 : historique et rollback

Objectif : revenir instantanément à une version stable depuis l'onglet History and Rollback.

Capture à ajouter

Fig. 22. Onglet History and Rollback listant les révisions déployées

Capture à ajouter

Fig. 23. Rollback vers la dernière révision stable

Capture à ajouter

Fig. 24. Application de nouveau Healthy après rollback

Résultat

À compléter après la démonstration.

Conclusion

À compléter.

Dépôt

- GitHub : github.com/KacemMathlouthi/tp-devops